

Artificial Intelligence

Name : Ganesh Machhindra Khilari

Roll No : 24092016

Class : IMCA-2

Introduction to Artificial Intelligence

What is AI?

AI = Making Computers Think and Learn Like Humans

Imagine you're teaching a child:

You show them many cat pictures → They learn to recognize cats

You explain rules of a game → They learn to play

You correct their mistakes → They improve

AI does the same with computers!

Simple Definition:

AI is teaching machines to do smart things that usually need human brains:

Recognizing faces in photos

Understanding spoken words

Playing chess

Recommending movies on Netflix

Driving cars automatically

❖ Defining AI Problems as State Space Search

Example: *Solving a Puzzle*

Current state: How the puzzle pieces are arranged RIGHT NOW

Next state: How they'll look after you make a move

Goal state: How the puzzle should look when solved

The Basic Idea

Analogy: *Finding Keys in Your House*

Problem: Lost your keys somewhere in the house

Initial state: You're in the living room, keys lost

Goal state: You found the keys

Possible moves: Go to kitchen, bedroom, bathroom, etc.

Solution: The sequence of rooms you check

Key Terms in above example :

Term	What It Means	Example
State	Current situation	I'm in the kitchen looking for keys
State Space	All possible situations	All rooms I could be in while searching
Initial State	Starting point	I start in living room
Goal State	What we want	Keys found!
Operators	Possible moves	Walk to kitchen, Open drawer

❖ Problem Characteristics:

Different problems need different approaches

A. Fully Observable vs Partially Observable

Fully: You see everything (like chess board)

Partial: You see only part (like poker - can't see opponents' cards)

B. Deterministic vs Stochastic

Deterministic: Same action → same result every time (like moving chess piece)

Stochastic: Same action → different results (like rolling dice)

C. Episodic vs Sequential

Episodic: Each decision independent (like diagnosing patients - each patient separate)

Sequential: Decisions affect future (like playing chess - each move affects next)

D. Discrete vs Continuous

Discrete: Limited choices (like chess - finite moves)

Continuous: Infinite choices (like steering wheel - infinite angles)

AI Problem Solving is in simple way as follows

1. Knowing where you are (State)
2. Knowing where you want to be (Goal)
3. Knowing what moves you can make (Operators)
4. Finding the best sequence of moves (Search)
5. Following the rules (Constraints)

❖ Heuristic search technique

What is "Heuristic"?

Think of a heuristic as a "**smart shortcut**"

Heuristic = Educated guess that helps find solutions faster

Different Heuristic search techniques

1. Generate-and-test
2. Hill Climbing
3. Best First Search
4. Problem Reduction
5. Constraint Satisfaction

Simple Example:

Problem: Find a restaurant in a new city

- **Without heuristic:** Check every street (takes hours)
- **With heuristic:** Look where cars are parked (smart guess)

Saves time and effort!

▪ **Different Heuristic search techniques as follows:**

1. Generate-and-Test

What It Is:

"Try random things until something works"

How It Works:

1. Generate a possible solution (make a guess)
2. Test if it's correct
3. Repeat until you find solution

Real-Life Example: Finding Lost Phone

Step 1: Generate a guess → "Maybe in bedroom?"

Step 2: Test → Go check bedroom

Step 3: Not there? Generate new guess → "Maybe in car?"

Step 4: Test → Go check car

Repeat until found!

2. Hill Climbing

What It Is:

"Always go uphill" (toward better solution)

Simple Analogy:

1. You're in Cloudy Mountain. You can only see your immediate area.
2. Check ground around you
3. Take a step toward higher ground
4. Keep going until no higher ground around

3. Best-First Search

What It Is:

"Always check the most promising option first"

Example: Finding Book in Library

- 1.Look at section labels (heuristic: section closeness to topic)
- 2.Check most relevant section first
- 3.If not found, check next most relevant

4. Problem Reduction

What It Is:

"Break big problem into smaller problems"

How It Works:

Original Problem: Build house

Reduce to:

1. Build foundation (subproblem)
2. Build walls (subproblem)
3. Build roof (subproblem)
4. Paint house (subproblem)

5. Constraint Satisfaction

What It Is:

"Find solution that follows ALL rules"

Analogy: Sudoku puzzle

Must satisfy:

You must choose a number that:

- is not in the same row
- not in the same column
- not in the same 3×3 box

❖ Knowledge Representation

What is Knowledge Representation?

"How to store facts in a computer's brain"

Knowledge Representation = Putting information in a format of computers can understand and use

Simple Example:

Human brain: Remembers "Cats eat fish"

Computer brain: Needs a way to store same fact so it can use it later

Approaches to Knowledge Representation

Four Main Ways:

1) Logical Approach

Uses logic rules

Example: "If it rains AND you go out, you get wet"

2) Network Approach

Network Approach (Web-like)

Connects ideas like mind map

Example: Animal → Mammal → Dog

3) Frame Approach

Like filling out a form

Dog:

Legs: 4

Sound: Bark

Type: Pet

4) Rule Approach

If-Then rules

Example: IF animal has feathers THEN it's a bird

❖ Game Playing:

The Challenge:

How to make computers play games smartly?

Think: Playing chess against a computer that never loses!

1. Minimax Algorithm

The Basic Idea:

Think ahead and assume your opponent plays perfectly

How Minimax Works:

Step 1: Look at all possible moves

Step 2: For each move, think what opponent will do

Step 3: Choose move with BEST worst-case outcome

2. Adding Alpha-Beta Cutoffs

The Problem in minmax :

Minimax checks EVERY possibility → TOO SLOW!

Example: Chess has billions of possibilities!

The Solution:

Skip checking branches that don't matter.

Alpha-Beta cutoffs are:

- **Alpha (α)** = Always getting better (for MAX)
- **Beta (β)** = Bad for opponent (good for MIN)
- **Cutoff** = Don't waste time on obvious options

Real-World Examples

1. Chess Computers:

- Without Alpha-Beta: Check 1 billion moves
- With Alpha-Beta: Check 1 million moves (1000x faster!)

➤ **When to Use What?**

i. Use Regular Minimax when:

- Game is simple (like Tic-Tac-Toe)
- Few possible moves
- You can check all possibilities

ii. Use Alpha-Beta Minimax when:

- Game is complex (chess, checkers)
- Many possible moves
- Need to search deep but fast

❖ Planning in AI

What is Planning?

Figuring out steps to reach a goal

Think: Making a to-do list for a robot

1. The Blocks World Example

The Classic AI Problem: Arrange blocks in a certain way

Setup:

- Blocks: A, B, C
- Table: Where blocks can sit
- Robot hand: Can pick up and move blocks

Rules:

- ❖ Can only pick up block if nothing on top of it
- ❖ Can only put block on table or on another block
- ❖ Hand can only hold one block at a time

Components of a Planning System:

4 Main Parts as follows

- A. Goals : What you can do
- B. Initial State : Where you start from
- C. Actions : What you can do
- D. Planner : The brain that figures out steps

❖ Learning in AI

What is Learning?

learning from experience rather than following newly written instructions

Think: Baby learning to walk

- Falls down 100 times
- Gets better each time
- No one "reprograms" the baby

In AI: Computer program improves by itself!

1. Rote Learning

What it is:

Memorizing without understanding

Like: Learning multiplication table

$$2 \times 1 = 2$$

$$2 \times 2 = 4$$

$$2 \times 3 = 6$$

You memorize, but don't understand multiplication concept

2. Learning by Taking Advice

What it is:

"Following instructions from experts"

Like: Cooking with recipe

- Expert chef writes recipe
- You follow steps
- You learn to make dish

How it works:

Expert : "Do this, then that, then this"

AI : Follows exactly

Result : Learns procedure

3. Learning in Problem Solving**What it is:**

"Learn from solving similar problems"

Like: Doing math homework

Solve problem 1 → Learn method

Solve problem 2 → Same method works!

Solve problem 3 → Now I'm fast!

4. Learning from Examples

What it is:

"Show many examples, find patterns"

How it works:

Step 1: Collect labeled examples

Cat pictures → Label "cat"

Dog pictures → Label "dog"

Step 2: Find patterns

Cats: Pointy ears, whiskers, small nose

Dogs: Floppy ears, long snout, big nose

Step 3: Make rules

IF pointy ears AND whiskers THEN cat

IF long snout AND floppy ears THEN dog

Step 4: Test on new pictures